1. What Is Machine Learning

Machine learning is a branch of artificial intelligence in which a computer system improves at a task by studying examples instead of following hand-written rules. A spam filter is a classic case. Rather than a programmer listing every suspicious phrase, the system is shown thousands of emails already labelled as spam or not spam, and it works out which patterns separate the two groups.

The core ingredients are data, a model, and a training process. The data describes the problem, the model is a mathematical function with adjustable parameters, and training is the procedure that adjusts those parameters so the model's predictions move closer to the correct answers. The quality and quantity of data usually matter more than the choice of algorithm.

A trained model is only useful if it generalizes, meaning it performs well on examples it has never seen. To check this, practitioners hold back part of the data as a test set. If a model scores highly on training data but poorly on the test set, it has memorized instead of learned, a problem known as overfitting.

2. Types of Learning

In supervised learning, every training example comes with a correct label, and the model learns to predict that label from the input. Predicting house prices from size and location is a regression task, while deciding whether a photo shows a cat or a dog is a classification task. Most commercial machine learning systems fall into this category.

Unsupervised learning works without labels. The goal is to discover structure that already exists in the data, such as grouping customers with similar buying habits through clustering, or compressing many features into a few meaningful ones through dimensionality reduction. Because no answer key exists, evaluating the results often requires human judgment.

Reinforcement learning is different from both. An agent interacts with an environment, takes actions, and receives rewards or penalties. Over many attempts it learns a strategy that maximizes total reward. This approach has produced game-playing systems and is also used to tune how language models respond to people.

3. Neural Networks and Embeddings

A neural network is built from layers of simple units, each of which multiplies its inputs by weights, adds them up, and passes the result through a nonlinear function. Stacking many layers lets the network represent very complicated relationships. Training uses an algorithm called backpropagation to nudge every weight in the direction that reduces the prediction error.

An embedding is a list of numbers that represents the meaning of an item, such as a word, a sentence, or an image. Models are trained so that items with similar meaning end up with similar numbers. The sentence 'a dog is running in the park' and 'a puppy sprints across the grass' should therefore produce vectors that lie close together.

Closeness between embeddings is usually measured with cosine similarity, which compares the direction of two vectors and ignores their length. A score near one means the vectors point the same way and the texts are semantically related, while a score near zero means they are essentially unrelated. This single idea powers modern semantic search.

4. Retrieval-Augmented Generation

Large language models know only what they absorbed during training, so they can be out of date and may invent details when they do not know an answer. Retrieval-augmented generation, or RAG, addresses this by fetching relevant passages from an external knowledge source at question time and giving them to the model as context alongside the question.

A basic RAG pipeline has two phases. In the indexing phase, documents are split into chunks, each chunk is converted into an embedding, and the embeddings are stored in a vector database. In the query phase, the user's question is embedded, the most similar chunks are retrieved, and the language model writes an answer grounded in those chunks.

The main benefit of RAG is that knowledge can be updated without retraining the model. Adding a new policy document only requires indexing it. RAG also makes answers easier to verify, because the system can show exactly which passages it relied on, which helps users trust the response and helps developers debug mistakes.

5. Chunking and Vector Search

Chunking determines how a document is cut into pieces before indexing, and it strongly affects retrieval quality. Chunks that are too small lose surrounding context and may contain half an idea, while chunks that are too large blend several topics together, which weakens the embedding and wastes space in the model's limited context window.

A common starting point is chunks of a few hundred words with a small overlap between neighbours, so that a sentence falling on a boundary still appears intact in at least one chunk. Splitting along natural boundaries such as paragraphs, headings, or sections usually works better than cutting at a fixed character count.

Once chunks are embedded, finding the closest ones to a query is a nearest-neighbour search. For small collections, comparing the query against every vector is fine. For millions of vectors, approximate methods such as HNSW graphs trade a little accuracy for a large gain in speed, and are built into most vector databases.

6. Evaluating a RAG System

A RAG system can fail in two separate places, and it helps to measure them separately. Retrieval can fail by returning irrelevant chunks or missing the right one, and generation can fail by ignoring the retrieved context or adding claims that the context does not support. Judging only the final answer hides which stage is responsible.

To evaluate retrieval, build a small set of test questions where you already know which chunk contains the answer. Then measure how often that chunk appears in the top results, using metrics such as recall at k or mean reciprocal rank. Even twenty carefully written questions will reveal weaknesses in chunking or in the embedding model.

To evaluate generation, check whether each statement in the answer can be traced back to the retrieved text, a property called faithfulness, and whether the answer actually addresses the question. Teams often combine manual review of a sample with automated scoring, and they re-run the same tests whenever they change the pipeline.